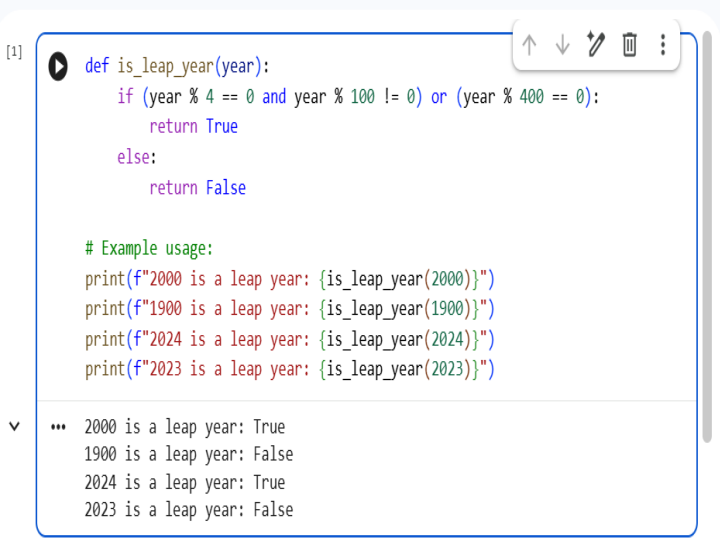
AI ASSISTED CODING

Lab Assignment-4.3

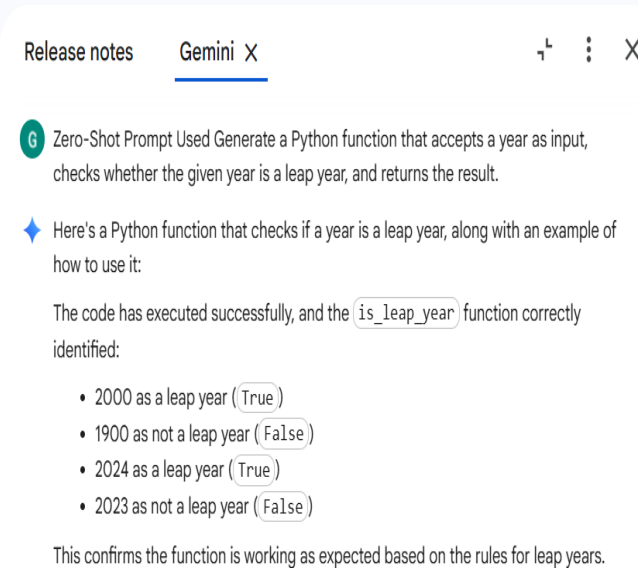
2303A51463

B-07

Task 1: Zero-Shot Prompting – Leap Year Check



```
[1]
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False

# Example usage:
print(f"2000 is a leap year: {is_leap_year(2000)}")
print(f"1900 is a leap year: {is_leap_year(1900)}")
print(f"2024 is a leap year: {is_leap_year(2024)}")
print(f"2023 is a leap year: {is_leap_year(2023)}")

2000 is a leap year: True
1900 is a leap year: False
2024 is a leap year: True
2023 is a leap year: False
```



Release notes Gemini X

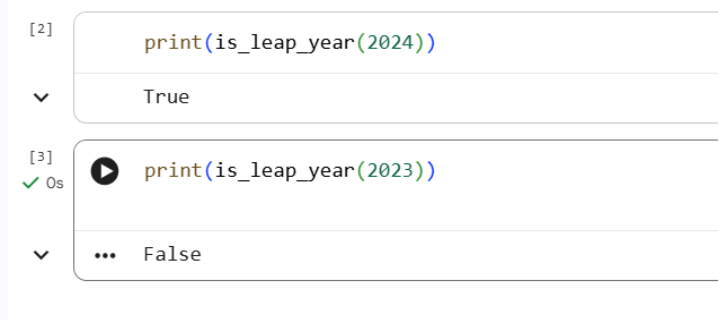
Zero-Shot Prompt Used Generate a Python function that accepts a year as input, checks whether the given year is a leap year, and returns the result.

Here's a Python function that checks if a year is a leap year, along with an example of how to use it:

The code has executed successfully, and the `is_leap_year` function correctly identified:

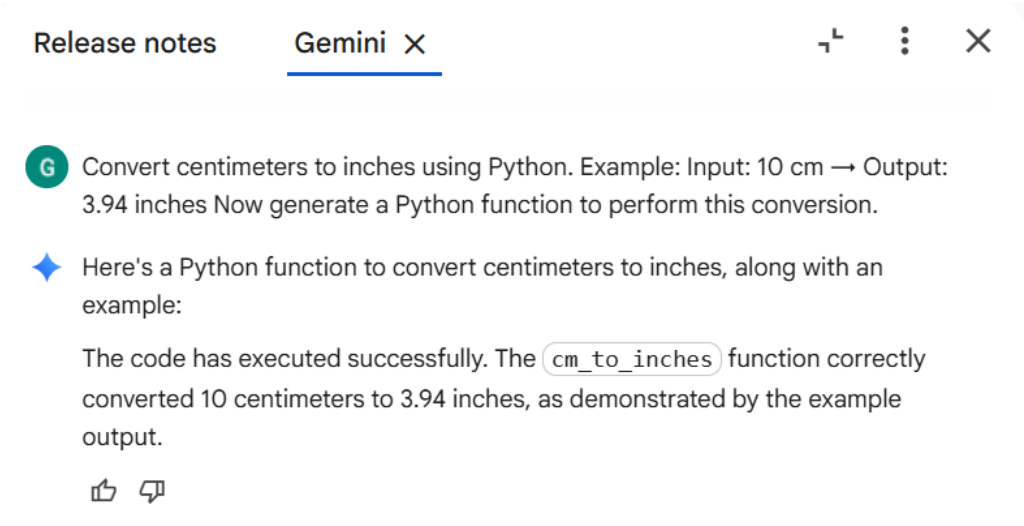
- 2000 as a leap year (`True`)
- 1900 as not a leap year (`False`)
- 2024 as a leap year (`True`)
- 2023 as not a leap year (`False`)

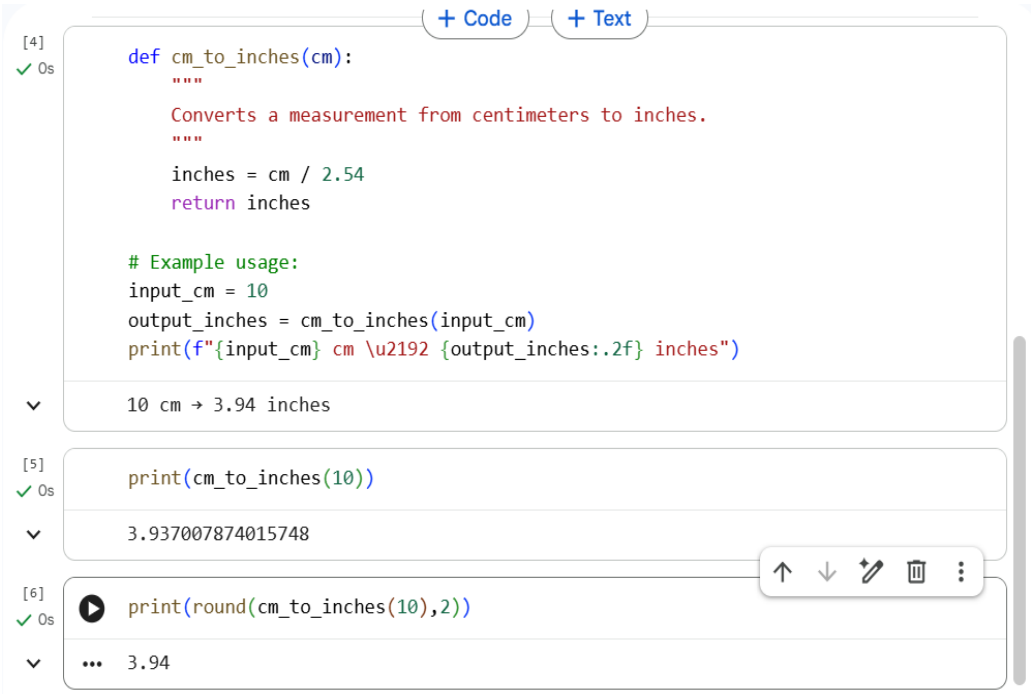
This confirms the function is working as expected based on the rules for leap years.



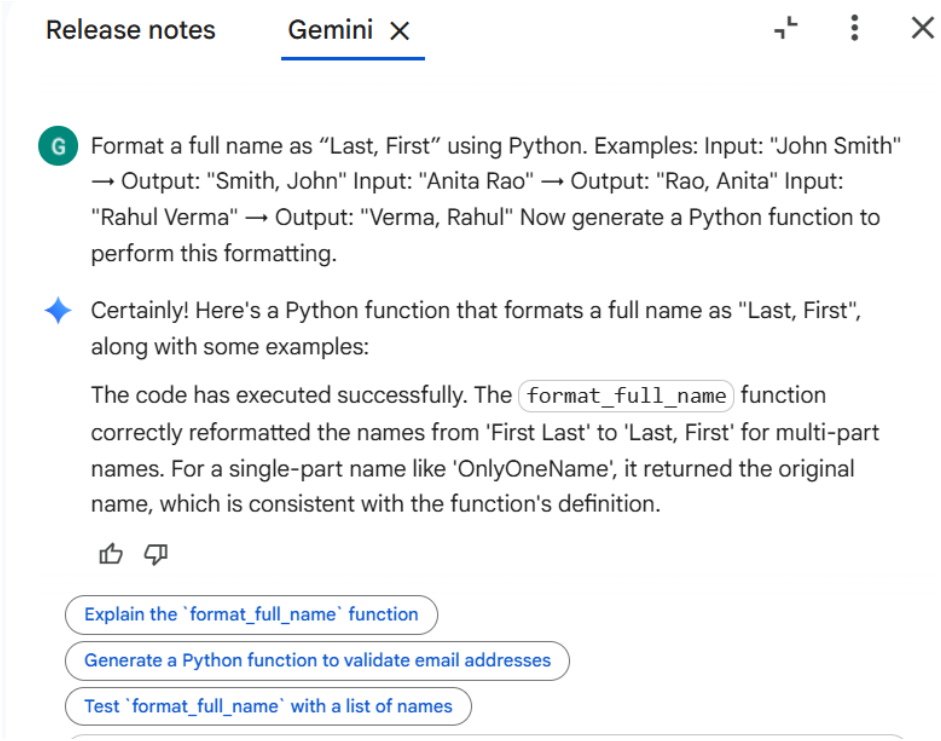
```
[2]
print(is_leap_year(2024))

True

[3]
✓ 0s
print(is_leap_year(2023))

False
```

Task 2: One-Shot Prompting – Centimeters to Inches Conversion





Task 3: Few-Shot Prompting – Name Formatting



[8]

```python
def format_full_name(full_name):

    name_parts = full_name.split()
    if len(name_parts) >= 2:
        first_name = name_parts[0]
        last_name = name_parts[-1]
        return f"{last_name}, {first_name}"
    else:
        return full_name # Return original name if not enough parts

# Example usage:
print(f"'John Smith' -> '{format_full_name("John Smith")}'")
print(f"'Anita Rao' -> '{format_full_name("Anita Rao")}'")
print(f"'Rahul Verma' -> '{format_full_name("Rahul Verma")}'")
print(f"'OnlyOneName' -> '{format_full_name("OnlyOneName")}'") # Edge case
```

```
'John Smith' -> 'Smith, John'
'Anita Rao' -> 'Rao, Anita'
'Rahul Verma' -> 'Verma, Rahul'
'OnlyOneName' -> 'OnlyOneName'
```

[10] ✓ 0s

```python
print(format_full_name("John Smith"))
```

```
Smith, John
```

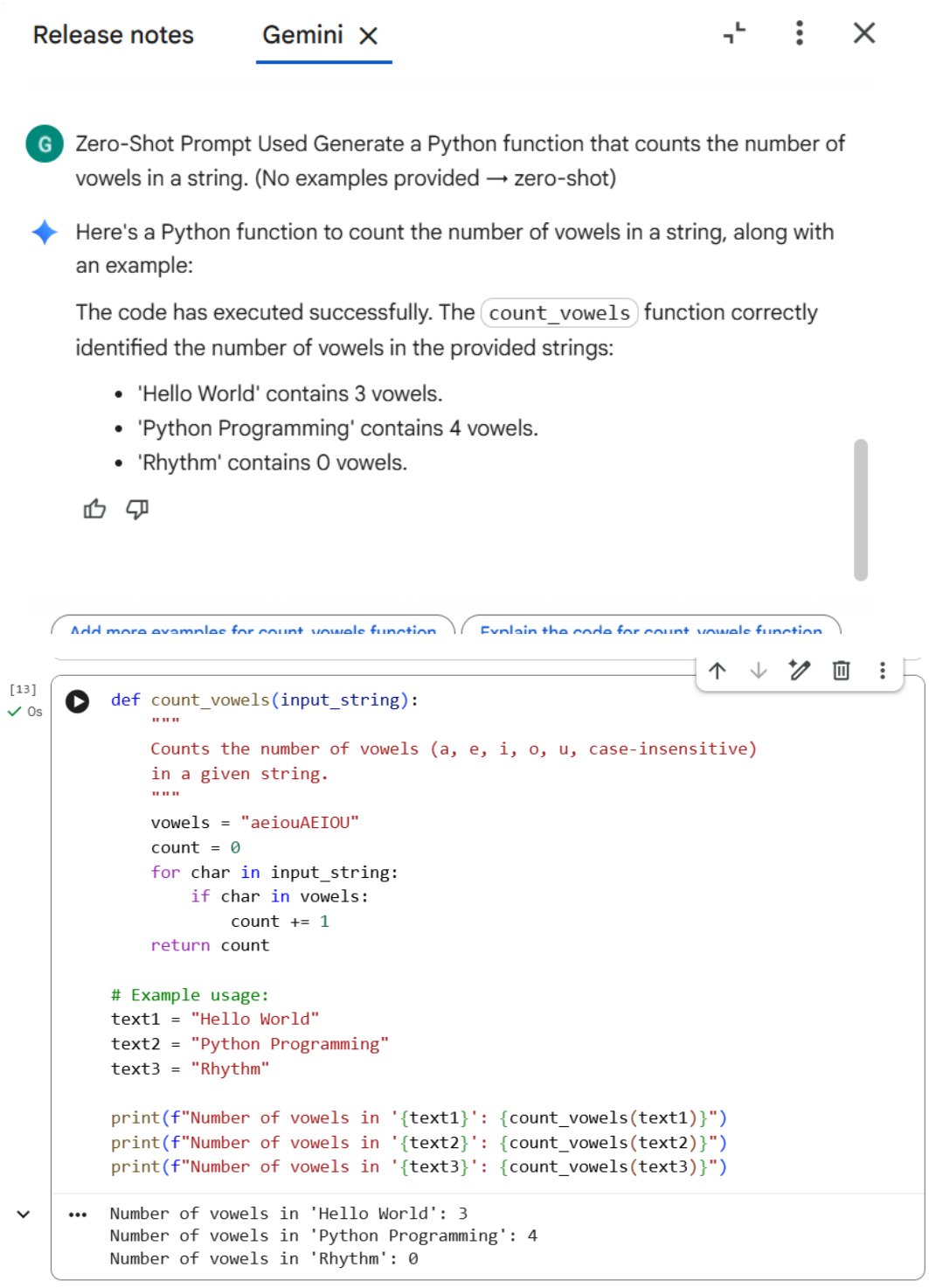
[11] ✓ 0s

```python
print(format_full_name("Anita Rao"))
```

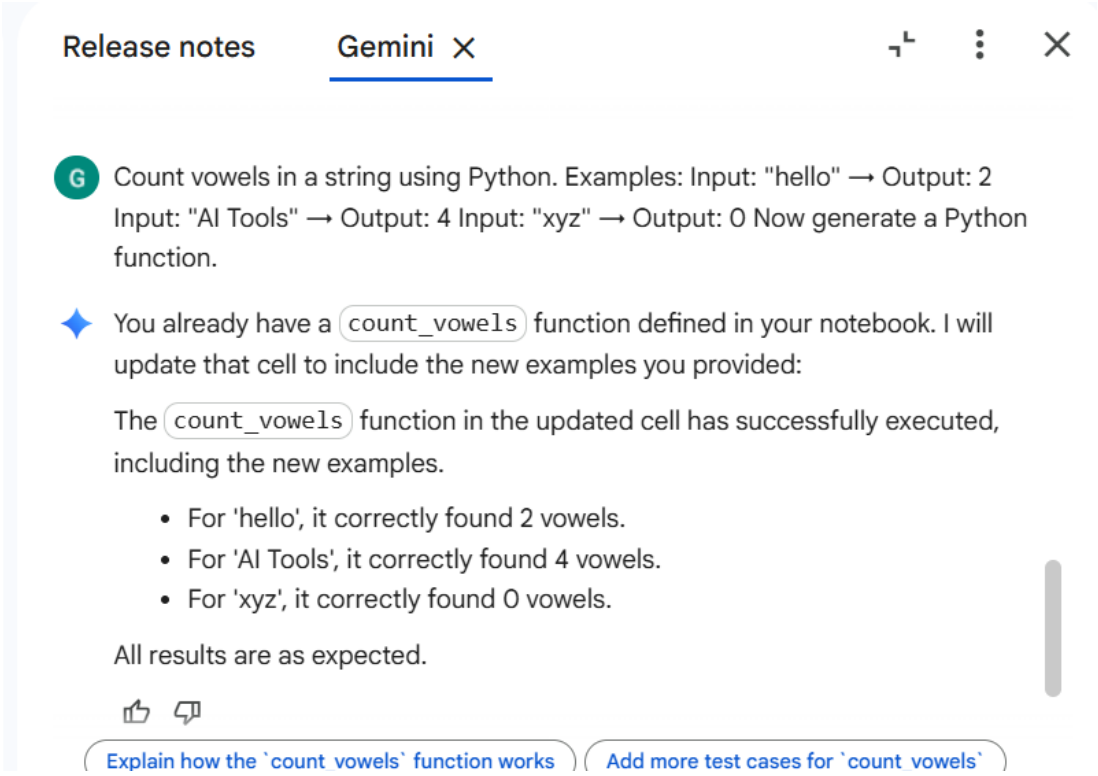
```
Rao, Anita
```

Task 4: Comparative Analysis – Zero-Shot vs Few-Shot

Zero-Shot:



Few-Shot:



[14]

```python
def count_vowels(input_string):
    vowels = "aeiouAEIOU"
    count = 0
    for char in input_string:
        if char in vowels:
            count += 1
    return count

# Example usage:
text1 = "Hello World"
text2 = "Python Programming"
text3 = "Rhythm"

print(f"Number of vowels in '{text1}': {count_vowels(text1)}")
print(f"Number of vowels in '{text2}': {count_vowels(text2)}")
print(f"Number of vowels in '{text3}': {count_vowels(text3)}")
print(f"Number of vowels in 'hello': {count_vowels('hello')}")
print(f"Number of vowels in 'AI Tools': {count_vowels('AI Tools')}")
print(f"Number of vowels in 'xyz': {count_vowels('xyz')}")
```

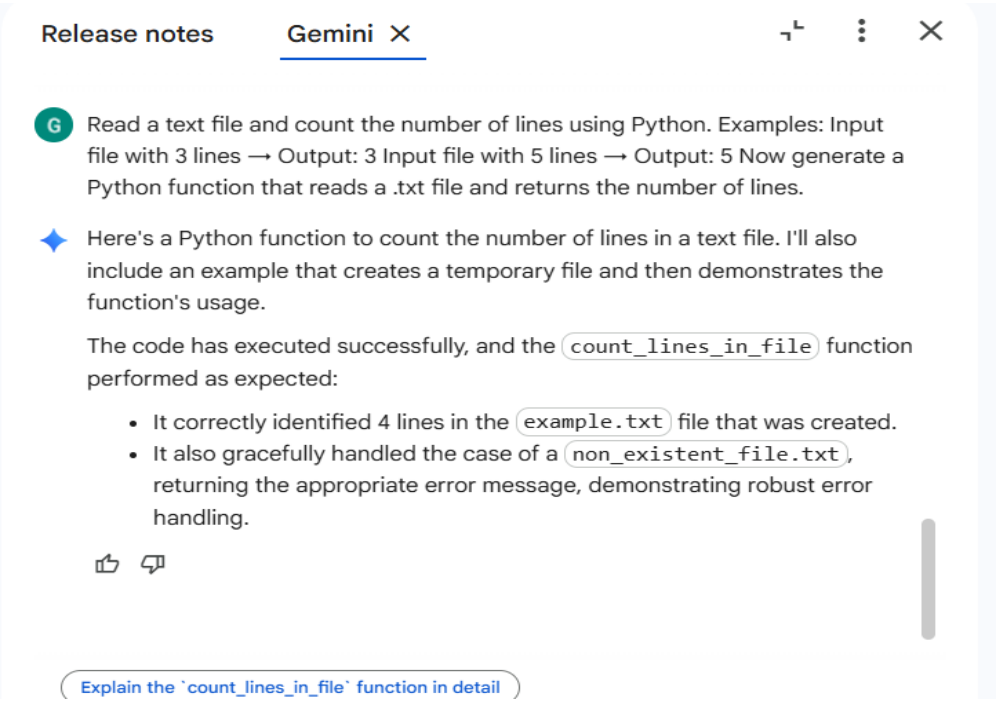
```
Number of vowels in 'Hello World': 3
Number of vowels in 'Python Programming': 4
Number of vowels in 'Rhythm': 0
Number of vowels in 'hello': 2
Number of vowels in 'AI Tools': 4
Number of vowels in 'xyz': 0
```

Comparison Table:

Criteria	Zero-Shot Output	Few-Shot Output
Accuracy	Correct	Correct
Readability	Moderate	High
Logical Clarity	Explicit loop	Concise & clear
Code Length	Longer	Shorter
Efficiency	Good	Better

- **Zero-shot prompting** produces correct but more basic logic
- **Few-shot prompting** improves:
- Code clarity
- Conciseness
- Professional style
- Few-shot prompting is **more effective for pattern-based and logic-refinement tasks**

Task 5: Few-Shot Prompting – File Handling



```python
def count_lines_in_file(filepath):
    try:
        with open(filepath, 'r') as file:
            line_count = sum(1 for line in file)
        return line_count
    except FileNotFoundError:
        return f"Error: File not found at {filepath}"
    except Exception as e:
        return f"An error occurred: {e}"

dummy_file_content = "This is line 1.\nThis is line 2.\nAnd this is line 3.\nLast line, line 4."
dummy_filepath = 'example.txt'

with open(dummy_filepath, 'w') as f:
    f.write(dummy_file_content)

print(f"Created '{dummy_filepath}' with the following content:\n---\n{dummy_file_content}\n---")
# Count lines in the dummy file
num_lines = count_lines_in_file(dummy_filepath)
print(f"Number of lines in '{dummy_filepath}': {num_lines}")
# Example with a file that doesn't exist
missing_file_path = 'non_existent_file.txt'
num_lines_missing = count_lines_in_file(missing_file_path)
print(f"Number of lines in '{missing_file_path}': {num_lines_missing}")
```

```
Created 'example.txt' with the following content:
---
This is line 1.
This is line 2.
And this is line 3.
Last line, line 4.
---
Number of lines in 'example.txt': 4
Number of lines in 'non_existent_file.txt': Error: File not found at non_existent_file.txt
```

Logic Explanation:

- The file is opened in **read mode.**
- `readlines()` reads all lines into a list.
- `len()` counts the number of lines.
- The function returns the total line count.
- `with open()` ensures the file is closed automatically.